SHORTZ-APP

PLANO DE GERENCIAMENTO DE CONFIGURAÇÃO

Projeto Web – Shortz-App

Versão 1.0

Campo	Descrição
Projeto	Shortz-App
Gerente de Projeto	Gustavo Henrique Forte
Gerente/Responsável de GCS-DevOps	Gustavo Henrique Forte
Desenvolvedor	Gustavo Henrique Forte
Repositório principal	https://github.com/gustavohenfor/Shortz-App.git

Histórico de Revisões

Data	Versão	Descrição	Autor
20/08/2026	1.0	Criação da aplicação.	Gustavo Henrique Forte
20/08	1.1	Desenvolvimento de Arquitetura MVC.	Gustavo Henrique Forte
21/08/2026	1.2	Desenvolvimento de Controllers e Routes.	Gustavo Henrique Forte
21/08/2026	2.0	Integração com a View.	Gustavo Henrique Forte
22/08/2026	2.1	Ajuste na view, na main.ejs.	Gustavo Henrique Forte
22/08/2026	2.2	Desenvolvendo a camada Service.	Gustavo Henrique Forte

Data	Versão	Descrição	Autor
22/08/2026	2.3	Ajustes finais na View e MVC.	Gustavo Henrique Forte

Sumário

Sumário 2
1. Introdução 3
1.1 Finalidade 3
1.2 Escopo 3
1.3 Definições, Acrônimos e Abreviações 3
1.4 Referências 4
1.5 Visão Geral 4
2. Objetivo do Sistema 4
3. Ferramentas e Ambiente de Desenvolvimento 6
4. Identificação da Configuração 6
4.1 Itens de Configuração 6
4.2 Convenção de Nomenclatura 7
4.3 Estrutura do Repositório 7
5. Integração Contínua (CI) 8
5.3 Critérios de Qualidade (Quality Gate) 9
6. Entrega e Implantação Contínua (CD) 9
7. Infraestrutura como Código (IaC) 11
7.1 Versionamento de Infraestrutura 11
8. Controle de Configuração e Mudança 12
8.1 Processo de Solicitação de Mudança 12
8.2 Comitê de Controle de Mudança (CCM) 13
9. Baselines e Releases 14
9.1 Geração Automática de Baseline 14
9.2 Notas de Versão (Release Notes) 14
10. Estimativa do Status de Configuração 15
10.1 Monitoramento e Observabilidade 15
10.2 Métricas DORA 16
10.3 Relatórios e Auditorias 16
11. Marcos 16
12. Treinamento e Recursos 17
13. Controle de Software de Subcontratados, Fornecedores e Dependências de Terceiros 18
14. Quadro-Resumo do Pipeline CI/CD 19

1. Introdução

O **Shortz-App** é uma aplicação web voltada ao compartilhamento de vídeos curtos, permitindo que os usuários publiquem, assistam e interajam com conteúdos disponíveis na plataforma.

O sistema possui funcionalidades como cadastro e autenticação de usuários, publicação e reprodução de vídeos, comentários, curtidas, perfis de usuários, seguidores, feed personalizado, playlists e busca por vídeos e usuários. Essas funcionalidades foram desenvolvidas progressivamente ao longo do projeto.

O projeto tem como objetivo proporcionar uma experiência de compartilhamento e interação com conteúdos em vídeo, permitindo também que os usuários organizem seus conteúdos e acompanhem outros usuários da plataforma.

1.1 Finalidade

A finalidade do Shortz-App é oferecer uma plataforma onde usuários possam **publicar, visualizar e interagir com vídeos**, além de acompanhar outros usuários e organizar conteúdos de seu interesse.

1.2 Escopo

O sistema contempla:

- **Cadastro e login de usuários;**
- **Gerenciamento de perfil;**
- **Upload e reprodução de vídeos;**
- **Feed de vídeos;**
- **Curtidas;**
- **Comentários;**
- **Seguidores e seguindo;**
- **Feed baseado em usuários seguidos;**
- **Playlists;**
- **Busca por vídeos e usuários;**
- **Exibição de perfis e vídeos publicados.**

As funcionalidades de seguidores, por exemplo, foram implementadas com uma relação entre usuários e um feed priorizado com base nas pessoas seguidas.

1.3 Definições, Acrônimos e Abreviações

Termo	Definição
Shortz-App	Aplicação web de compartilhamento de vídeos curtos.
Usuário	Pessoa cadastrada que utiliza as funcionalidades da plataforma.
Vídeo	Conteúdo publicado por um usuário na plataforma.
Playlist	Conjunto de vídeos organizado pelo usuário.
Feed	Área da aplicação que apresenta vídeos ao usuário.
Like	Interação utilizada para indicar que o usuário gostou de um vídeo.
Follow	Relação utilizada para que um usuário acompanhe outro usuário.
EJS	Tecnologia utilizada para criação das páginas dinâmicas da aplicação.

Termo	Definição
Sequelize	ORM utilizado para comunicação entre a aplicação e o banco de dados.
MySQL	Banco de dados utilizado para armazenar as informações da aplicação.

1.4 Referências

- Documentação e código-fonte do Shortz-App, utilizado como base para aplicação das práticas de gerenciamento de configuração.

1.5 Visão Geral

O Shortz-App é organizado em diferentes funcionalidades que permitem ao usuário interagir com vídeos e outros usuários da plataforma.

O sistema possibilita o **cadastro e autenticação de usuários**, gerenciamento de perfil, publicação e reprodução de vídeos, além de funcionalidades de interação como **curtidas e comentários**.

Também fazem parte da aplicação recursos sociais, como **seguir outros usuários e visualizar um feed baseado nesses relacionamentos**. O projeto possui ainda funcionalidades de **playlists**, permitindo organizar vídeos, e páginas de perfil para visualizar informações e conteúdos publicados pelos usuários.

A aplicação foi desenvolvida de forma incremental, com novas funcionalidades sendo adicionadas ao longo das etapas do projeto. Por exemplo, o sistema de seguidores adicionou uma relação entre usuários e um feed priorizado, enquanto a página de perfil passou a apresentar informações do usuário e seus vídeos.

2. Objetivo do Sistema

O objetivo do **Shortz-App** é oferecer uma plataforma web para **compartilhamento e interação com vídeos curtos**, permitindo que os usuários publiquem seus próprios conteúdos e consumam vídeos de outros usuários.

A aplicação busca proporcionar uma experiência de interação entre os usuários por meio de funcionalidades como **curtidas, comentários, seguidores e playlists**, além de permitir que cada usuário possua um perfil com seus próprios conteúdos.

O sistema também possui um **feed de vídeos**, incluindo a possibilidade de priorizar conteúdos de usuários que são seguidos, proporcionando uma experiência mais personalizada.

2.2 Público-alvo

O Shortz-App é destinado a **usuários interessados em compartilhar e consumir vídeos curtos**, permitindo que eles publiquem seus próprios conteúdos, acompanhem outros usuários e interajam com os vídeos disponíveis na plataforma.

O sistema também pode ser utilizado como uma plataforma acadêmica para demonstrar o desenvolvimento de uma aplicação web completa, envolvendo funcionalidades de autenticação, gerenciamento de usuários, vídeos e interações sociais.

2.3 Principais Funcionalidades

As principais funcionalidades do Shortz-App são:

- **Cadastro e login:** permite que usuários criem suas contas e realizem autenticação.
- **Perfil de usuário:** permite visualizar informações do usuário e os vídeos publicados por ele.
- **Upload de vídeos:** permite que usuários publiquem vídeos na plataforma.
- **Reprodução de vídeos:** permite assistir aos vídeos publicados.
- **Curtidas:** permite que usuários curtam e removam curtidas de vídeos.
- **Comentários:** permite adicionar e visualizar comentários.
- **Seguidores:** permite seguir e deixar de seguir outros usuários.
- **Feed personalizado:** apresenta conteúdos relacionados aos usuários seguidos.
- **Playlists:** permite organizar vídeos em playlists.
- **Busca:** permite localizar vídeos e usuários na plataforma.

2.4 Requisitos Funcionais

Podemos transformar essas funcionalidades em requisitos:

ID	Requisito
RF01	O sistema deve permitir o cadastro de usuários.
RF02	O sistema deve permitir que usuários realizem login.
RF03	O sistema deve permitir editar informações do perfil.
RF04	O sistema deve permitir realizar upload de vídeos.
RF05	O sistema deve permitir reproduzir vídeos publicados.
RF06	O sistema deve permitir curtir e remover curtidas de vídeos.
RF07	O sistema deve permitir adicionar comentários aos vídeos.
RF08	O sistema deve permitir seguir e deixar de seguir usuários.
RF09	O sistema deve apresentar um feed de vídeos.
RF10	O sistema deve permitir criar e gerenciar playlists.
RF11	O sistema deve permitir pesquisar vídeos e usuários.
RF12	O sistema deve permitir visualizar o perfil e os vídeos publicados por outros usuários.

3. Ferramentas e Ambiente de Desenvolvimento

Para o desenvolvimento do Shortz-App, foram utilizadas ferramentas para edição do código, controle de versão e armazenamento do projeto.

3.1 Visual Studio Code

O **Visual Studio Code (VS Code)** foi utilizado como ambiente de desenvolvimento para criação e edição dos arquivos do Shortz-App, incluindo os arquivos JavaScript, EJS, configurações e demais componentes da aplicação.

3.2 Git

O **Git** foi utilizado para realizar o controle de versão do projeto, permitindo registrar as alterações realizadas durante o desenvolvimento e manter um histórico das modificações.

3.3 GitHub

O **GitHub** foi utilizado para hospedar o repositório do Shortz-App, permitindo armazenar o código do projeto e acompanhar seu desenvolvimento por meio do histórico de versões.

3.4 Tecnologias utilizadas

Além das ferramentas de desenvolvimento, o projeto foi desenvolvido utilizando:

- **Node.js** — ambiente de execução da aplicação;
- **Express.js** — framework utilizado no desenvolvimento do servidor;
- **JavaScript** — linguagem principal da aplicação;
- **EJS** — utilizado para criação das páginas dinâmicas;
- **Sequelize** — ORM utilizado para comunicação com o banco de dados;
- **MySQL/MariaDB** — banco de dados utilizado pela aplicação.

3.5 Ambiente de desenvolvimento

O desenvolvimento foi realizado em ambiente local, utilizando o **Visual Studio Code**, com o projeto versionado por meio do **Git** e armazenado no **GitHub**. O servidor da aplicação é executado utilizando Node.js e a aplicação realiza a comunicação com o banco de dados relacional.

4. Identificação da Configuração

4.1 Itens de Configuração

Item	Descrição
Código-fonte	Arquivos JavaScript responsáveis pela aplicação, incluindo Models, Controllers, Services e Routes.
Views	Arquivos EJS utilizados na interface da aplicação.
Testes	Arquivos relacionados aos testes automatizados do sistema.
Banco de dados	Estruturas e configurações relacionadas ao banco utilizado pela aplicação.
Configurações	Arquivos necessários para configurar a execução do sistema, como package.json e arquivos de configuração.
Dependências	Bibliotecas utilizadas pelo projeto e registradas no package.json e arquivo de lock.
Documentação	Documentos relacionados ao projeto e à sua utilização.

4.2 Convenção de Nomenclatura

Item	Convenção	Exemplo
Branch de feature	feature/<ticket>-<descrição-curta>	feature/SIQQ-123-login-oauth
Commit	Conventional Commits: <tipo>(<escopo>): <descrição>	feat(auth): adiciona login via OAuth2
Tag de release	Versionamento Semântico vX.Y.Z	v2.3.0
Imagem de container	<registro>/<projeto>:<versão|sha>	ghcr.io/org/projeto:2.3.0
...	...	...

4.3 Estrutura do Repositório

```
Shortz-App/
 |
 ├── bin/
 ├── config/
 ├── middlewares/
 ├── modules/
 |    ├── user/
 |    ├── video/
 |    ├── comment/
 |    ├── playlist/
 |    ├── follow/
 |
 |
 ├── public/
 ├── views/
 ├── tests/
```

```
├── uploads/
│
├── app.js
├── package.json
├── package-lock.json
└── .env
```

5. Integração Contínua (CI)

A Integração Contínua será utilizada no **Shortz-App** como prática para verificar automaticamente as alterações realizadas no código antes de sua integração à branch principal.

O processo será baseado no **Git e GitHub**, utilizando Pull Requests e um pipeline automatizado para executar verificações e testes do projeto.

O objetivo é identificar erros antecipadamente e garantir que alterações realizadas por um integrante não comprometam funcionalidades existentes do sistema.

5.1 Etapas do Pipeline de CI

Para o Shortz-App, propõe-se o seguinte fluxo:

Pull Request → Instalação das dependências → Verificação → Testes → Resultado do Pipeline → Merge

1. Checkout do código

O pipeline obtém a versão do código presente no repositório para iniciar a execução.

2. Instalação das dependências

As dependências definidas no package.json serão instaladas para preparar o ambiente de execução do projeto.

3. Verificação do projeto

Será realizada uma verificação para identificar possíveis problemas no código antes da execução dos testes.

4. Execução dos testes

O pipeline executará os testes automatizados desenvolvidos para o Shortz-App.

Isso é especialmente adequado ao nosso projeto porque os roteiros já orientam a realização de testes das funcionalidades, incluindo cadastro, login, perfil, upload, streaming, likes e comentários.

5. Resultado do pipeline

Caso todas as etapas sejam concluídas corretamente, o pipeline será considerado **aprovado**.

Caso alguma etapa falhe, a alteração deverá ser corrigida antes de ser integrada à branch principal.

Observação: o template original inclui também lint, análise SAST/SCA e publicação de artefatos. Para o nosso Shortz, não precisamos afirmar que essas etapas já existem. Podemos adicioná-las como melhorias futuras caso decidamos realmente implementar.

5.2 Política de Pull Request e Revisão de Código

As alterações do Shortz-App deverão ser realizadas em uma branch específica e posteriormente submetidas por meio de **Pull Request** para a branch principal.

O fluxo proposto é:

Branch de funcionalidade → Pull Request → Revisão → CI → Merge

Os Pull Requests devem:

- apresentar uma descrição clara da alteração;
- possuir alterações relacionadas a uma funcionalidade ou correção específica;
- passar pelo pipeline de CI;
- ser revisados por outro integrante quando o projeto for desenvolvido em equipe;
- somente ser integrados após a aprovação do pipeline.

O template recomenda justamente que o merge somente seja permitido quando o pipeline estiver aprovado e que os PRs sejam pequenos para facilitar a revisão.

5.3 Critérios de Qualidade (Quality Gate)

Para que uma alteração seja integrada à branch principal do Shortz-App, serão considerados os seguintes:

Critério	Condição
Execução do projeto	Deve concluir sem erro na etapa de preparação
Testes automatizados	Todos os testes obrigatórios devem passar
Erros bloqueantes	Não devem existir erros que impeçam a execução da aplicação
Revisão	O Pull Request deve ser revisado quando houver outro integrante disponível
CI	O pipeline deve finalizar com sucesso

6. Entrega e Implantação Contínua (CD)

A Entrega e Implantação Contínua será utilizada como uma proposta para automatizar a disponibilização de novas versões do **Shortz-App** após a aprovação das alterações realizadas no processo de Integração Contínua.

A ideia é que uma alteração aprovada no código possa avançar de forma controlada para um ambiente de homologação e, posteriormente, para produção, mantendo a mesma versão que foi validada no processo de CI.

Esse princípio está de acordo com o modelo fornecido, que propõe que o CD promova a mesma versão validada pelo CI através dos ambientes, sem recompilar o artefato.

6.1 Etapas do Pipeline de CD

Para o Shortz-App, propõe-se o seguinte fluxo:

Merge na branch principal → Deploy em Homologação → Testes → Aprovação → Deploy em Produção → Verificação

1. Deploy em Homologação

Após a aprovação do Pull Request e conclusão do pipeline de CI, a versão do Shortz-App será disponibilizada em um ambiente de homologação.

Esse ambiente permitirá verificar o funcionamento da aplicação antes de disponibilizá-la aos usuários.

2. Testes de Aceitação

No ambiente de homologação serão executados testes para verificar se as principais funcionalidades continuam funcionando corretamente.

Entre eles:

- Login e cadastro;
- Perfil;
- Upload e reprodução de vídeos;
- Curtidas;
- Comentários;
- Seguidores;
- Playlists.

Essas funcionalidades já fazem parte do escopo de testes do Shortz-App, conforme os roteiros do projeto.

3. Aprovação para Produção

Caso os testes sejam concluídos com sucesso, a versão poderá ser aprovada para implantação em produção.

4. Deploy em Produção

Após a aprovação, a mesma versão validada em homologação será disponibilizada no ambiente de produção.

5. Verificação após implantação

Após o deploy, serão realizadas verificações básicas para confirmar que a aplicação está funcionando corretamente.

6.2 Estratégia de Implantação

Para o Shortz-App, a estratégia escolhida será inicialmente uma **implantação convencional**, na qual uma nova versão é disponibilizada após ser validada no ambiente de homologação.

Essa estratégia é adequada ao contexto acadêmico e ao porte atual do projeto, evitando a complexidade de estratégias como Canary Release ou Blue-Green.

O template apresenta diferentes estratégias, incluindo **Rolling Update, Blue-Green, Canary Release e Feature Flags**.

No futuro, caso o sistema cresça e passe a atender um número maior de usuários, estratégias como Blue-Green ou Canary poderiam ser consideradas para reduzir os riscos durante novas implantações.

6.3 Rollback e Recuperação

Em caso de falha após uma implantação, o Shortz-App deverá permitir o retorno à **última versão estável**.

O histórico mantido pelo Git/GitHub permite identificar versões anteriores do código e recuperar uma versão que tenha apresentado funcionamento adequado.

O processo proposto será:

Nova versão apresenta falha → Identificação do problema → Retorno à versão estável → Correção → Novo processo de CI/CD

O conceito segue a orientação do template de que uma implantação deve ser reversível e que a versão anterior deve permanecer disponível para permitir recuperação em caso de falha.

7. Infraestrutura como Código (IaC)

No contexto do **Shortz-App**, a utilização de IaC pode contribuir para padronizar o ambiente necessário para executar a aplicação, incluindo o servidor da aplicação, banco de dados e demais configurações necessárias.

7.1 Versionamento de Infraestrutura

Caso seja adotada uma solução de IaC para o Shortz-App, seus arquivos deverão ser armazenados no mesmo repositório GitHub do projeto ou em um repositório relacionado à infraestrutura.

Dessa forma, alterações na infraestrutura também poderão ser controladas pelo Git, permitindo identificar:

- quem realizou a alteração;
- quando ela foi realizada;
- qual configuração foi modificada;
- qual versão da infraestrutura está sendo utilizada.

Assim, a infraestrutura passa a seguir o mesmo princípio de controle de configuração aplicado ao código do Shortz-App.

7.2 Ambientes Efêmeros

Os arquivos de infraestrutura poderão ser integrados ao processo de **CI/CD** definido para o Shortz-App.

O fluxo proposto seria:

Alteração da infraestrutura → Git → Pull Request → Revisão → CI → Aprovação → Aplicação da alteração

Isso permite que alterações na infraestrutura sejam verificadas antes de serem aplicadas ao ambiente do Shortz-App.

8. Controle de Configuração e Mudança

8.1 Processo de Solicitação de Mudança

Para o Shortz-App, as mudanças podem surgir da necessidade de:

- adicionar uma nova funcionalidade;
- corrigir um erro;
- melhorar uma funcionalidade existente;
- alterar a interface;
- modificar uma consulta ou regra de negócio;
- alterar a estrutura do banco de dados.

Para mudanças comuns, será utilizado o seguinte fluxo:

Solicitação → Branch → Desenvolvimento → Commit → Pull Request → Revisão → CI → Merge

Exemplo no Shortz-App

Supondo que seja identificada a necessidade de **adicionar uma nova funcionalidade às playlists**:

1. A mudança é identificada e registrada;
2. É criada uma branch específica para a alteração;
3. O desenvolvimento é realizado;
4. As alterações são registradas por meio de commits;
5. É aberto um Pull Request no GitHub;
6. O código é revisado;
7. O pipeline de CI executa as verificações e testes;
8. Se os critérios forem atendidos, a alteração é integrada à branch principal.

Dessa forma, a alteração fica registrada no histórico do GitHub, permitindo identificar **o que foi alterado, quando foi alterado e qual código foi responsável pela mudança**.

Esse fluxo também evita que alterações sejam realizadas diretamente na versão principal do Shortz-App sem passar pelas verificações definidas no plano.

Mudanças no banco de dados

Alterações que envolvam o banco de dados merecem atenção especial.

Por exemplo:

Adicionar um novo campo ao cadastro de usuários.

Nesse caso, a alteração deve ser desenvolvida, testada e revisada antes de ser incorporada ao projeto, pois mudanças no banco podem afetar os **Models, Services, Controllers e demais funcionalidades que utilizam aquela informação**.

8.2 Comitê de Controle de Mudança (CCM)

Para o contexto atual do Shortz-App, não é necessário utilizar um Comitê de Controle de Mudança formal para todas as alterações.

Mudanças comuns, como correção de bugs ou implementação de funcionalidades menores, poderão ser controladas por meio de **Pull Requests e revisão de código**.

Entretanto, mudanças consideradas de **alto impacto ou maior risco** deverão receber uma análise mais cuidadosa antes de serem integradas.

Podem ser consideradas mudanças de alto impacto no Shortz-App:

- alteração significativa na estrutura do banco de dados;
- alteração no sistema de autenticação;
- mudança que afete várias funcionalidades;
- alteração que possa causar perda ou inconsistência de dados;
- mudança emergencial (**hotfix**) em uma versão em produção.

O documento também estabelece que o CCM seja acionado principalmente em mudanças de alto impacto, como alterações estruturais de infraestrutura, mudanças de esquema do banco em produção ou exceções ao fluxo normal. Para mudanças rotineiras, a revisão do Pull Request substitui a necessidade de um comitê formal.

Aplicação no Shortz-App

Podemos resumir assim:

Tipo de mudança	Processo
Correção simples de bug	Branch → PR → CI → Merge
Nova funcionalidade	Branch → PR → Revisão → CI → Merge
Alteração de interface	Branch → PR → CI → Merge
Alteração importante no banco	PR → Revisão mais criteriosa → CI → Aprovação
Alteração crítica em produção	Análise especial/CCM → Aprovação → Deploy

Assim, o Shortz-App mantém um processo **simples para mudanças comuns**, mas possui um nível maior de controle quando uma alteração pode comprometer partes importantes do sistema.

9. Baselines e Releases

9.1 Geração Automática de Baseline

Uma baseline do Shortz-App representa uma versão do sistema que foi considerada estável após passar pelas verificações e testes definidos no projeto.

A criação de uma baseline deverá ocorrer após uma alteração ou conjunto de alterações ser integrado à versão principal e apresentar funcionamento adequado.

No Git, essa versão pode ser identificada por meio de uma **tag**, permitindo relacionar uma identificação àquele estado específico do projeto.

Por exemplo:

Shortz-App v1.0.0

Essa versão poderá representar uma determinada etapa estável do Shortz-App.

A partir dela, novas funcionalidades podem ser desenvolvidas sem perder a referência da versão anterior.

Exemplo aplicado ao Shortz-App

Imagine que uma versão do projeto possua:

- cadastro e login;
- perfil;
- publicação de vídeos;
- reprodução de vídeos;
- curtidas;
- comentários.

Depois que essas funcionalidades forem testadas e consideradas estáveis, essa versão poderá ser registrada como uma baseline.

Posteriormente, funcionalidades como **seguidores ou playlists** podem ser desenvolvidas em novas alterações, mantendo a versão anterior como referência.

9.2 Notas de Versão (Release Notes)

Para organizar as versões do Shortz-App, poderá ser utilizado o Versionamento Semântico (SemVer):

MAJOR.MINOR.PATCH

Por exemplo:

v1.0

v1.1

v1.2

A lógica aplicada ao Shortz seria:

- **MAJOR:** alteração significativa que possa representar uma nova versão do sistema;
- **MINOR:** adição de novas funcionalidades sem quebrar as funcionalidades existentes;
- **PATCH:** correções ou pequenos ajustes.

Exemplo

Se o Shortz-App estiver na versão:

v1.0

e for adicionada uma nova funcionalidade importante, como **playlists**, uma nova versão poderia ser:

v1.1

Caso posteriormente seja corrigido um problema na funcionalidade de playlists:

v1.2

Esses exemplos são **propostos para o gerenciamento do Shortz-App**, não significam que essas versões tenham sido oficialmente criadas no histórico atual do projeto.

10. Estimativa do Status de Configuração

10.1 Monitoramento e Observabilidade

No Shortz-App, o monitoramento tem como objetivo identificar problemas que possam ocorrer durante a execução da aplicação e após novas alterações.

Como proposta de aplicação das práticas DevOps, o sistema poderá utilizar:

- **Logs da aplicação**, para identificar erros durante a execução;
- **Testes automatizados**, para verificar se as funcionalidades continuam funcionando;
- **Health checks**, para verificar se a aplicação está disponível;
- **Monitoramento de erros**, permitindo identificar problemas após uma nova versão;
- **Métricas de execução**, caso o sistema seja disponibilizado em um ambiente de produção.

No contexto atual do projeto, o foco principal será a utilização dos **testes e logs** para identificar problemas no Shortz-App.

Por exemplo, após uma alteração no sistema de **playlists**, os testes poderão verificar se a criação, edição e visualização das playlists continuam funcionando corretamente.

O documento do professor recomenda que ambientes de homologação e produção sejam monitorados por meio de métricas, logs e alertas para identificar rapidamente falhas após uma implantação.

10.2 Métricas DORA

Métrica	Aplicação no Shortz-App
Deployment Frequency	Quantidade de vezes que novas versões do Shortz-App são disponibilizadas.
Lead Time for Changes	Tempo entre uma alteração ser realizada no código e sua disponibilização.
Change Failure Rate	Percentual de alterações que resultam em problemas após a implantação.
MTTR	Tempo necessário para recuperar o Shortz-App após uma falha.

10.3 Relatórios e Auditorias

O histórico do **Git e GitHub** será utilizado como principal fonte de informações para acompanhar as alterações realizadas no Shortz-App.

Entre as informações que poderão ser verificadas estão:

- commits realizados;
- alterações no código;
- Pull Requests;
- revisões;
- resultados dos testes;
- versões e tags;
- releases;
- alterações relacionadas às funcionalidades.

Por exemplo, caso seja necessário descobrir quando uma alteração no sistema de **seguidores** foi realizada, será possível consultar o histórico do GitHub e identificar o commit ou Pull Request correspondente.

Além disso, os registros dos pipelines de CI/CD, quando implementados, poderão fornecer informações sobre **builds, testes e deploys**. O próprio modelo do professor considera esses registros e o histórico do versionamento como os principais insumos para auditoria da Gestão de Configuração, reduzindo a necessidade de relatórios manuais.

11. Marcos

Os marcos do Shortz-App representam etapas importantes na evolução do sistema e na adoção das práticas de Gerenciamento de Configuração e DevOps.
Esses marcos permitem acompanhar tanto o desenvolvimento das funcionalidades do sistema quanto a evolução dos processos utilizados para controlar, testar e disponibilizar suas versões.

11.1 Marcos do Shortz-App

Marco	Descrição
M1 — Estrutura inicial	Criação da estrutura inicial do Shortz-App e configuração do ambiente de desenvolvimento.
M2 — Autenticação	Implementação das funcionalidades de cadastro, login e controle de acesso dos usuários.
M3 — Gerenciamento de vídeos	Implementação do upload, armazenamento e reprodução dos vídeos.
M4 — Interações	Implementação de curtidas e comentários nos vídeos.
M5 — Perfis e seguidores	Implementação dos perfis de usuários, sistema de seguidores e feed priorizado.
M6 — Playlists	Implementação da criação e gerenciamento de playlists para organização dos vídeos.
M7 — Testes	Desenvolvimento e execução dos testes do Shortz-App, incluindo testes unitários, de integração e E2E.
M8 — Versão estável	Consolidação das principais funcionalidades e definição de uma versão estável do sistema.

As funcionalidades de seguidores e feed, por exemplo, fazem parte da evolução do Shortz-App e foram desenvolvidas utilizando Model, Service, Controller e Views EJS.

12. Treinamento e Recursos

12.1 Ferramentas

As principais ferramentas utilizadas ou propostas para o gerenciamento do Shortz-App são:

- **Visual Studio Code:** ambiente utilizado para desenvolvimento e manutenção do código;
- **Git:** controle de versão do projeto;
- **GitHub:** armazenamento do repositório, Pull Requests e acompanhamento das alterações;
- **Node.js:** execução da aplicação;
- **npm:** gerenciamento das dependências do projeto;
- **MySQL/MariaDB:** armazenamento dos dados da aplicação;
- **Ferramentas de testes:** utilizadas para verificar o funcionamento das funcionalidades do Shortz-App.

Para a evolução das práticas DevOps, também poderá ser utilizada uma ferramenta de **CI/CD**, como o GitHub Actions, permitindo automatizar testes e posteriormente o processo de entrega.

12.2 Conhecimentos necessários

Para trabalhar no Shortz-App, os integrantes devem possuir conhecimento básico sobre:

- JavaScript;
- Node.js;
- Express.js;
- EJS;
- Sequelize;
- MySQL/MariaDB;
- Git e GitHub;
- Controllers, Services, Models e Routes;
- testes automatizados;
- Pull Requests;
- versionamento;
- conceitos de CI/CD.

Além do conhecimento de desenvolvimento, os integrantes deverão compreender o fluxo de gerenciamento de mudanças definido neste plano.

12.3 Treinamento em DevOps

Para aplicar as práticas propostas ao Shortz-App, recomenda-se que os integrantes sejam capacitados nos seguintes conceitos:

Git/GitHub → Branches → Pull Requests → CI → Testes → CD → Releases

Também deverão compreender conceitos como:

- Integração Contínua;
- Entrega Contínua;
- Controle de configuração;
- Baselines;
- Versionamento Semântico;
- Rollback;
- Métricas DORA;
- Infraestrutura como Código.

O objetivo não é exigir que todos os integrantes sejam especialistas em DevOps, mas garantir que saibam utilizar o processo definido para o Shortz-App.

13. Controle de Software de Subcontratados, Fornecedores e Dependências de Terceiros

13.1 Gerenciamento das Dependências

As dependências utilizadas pelo Shortz-App deverão ser adicionadas ao projeto por meio do npm e registradas no repositório GitHub.

Entre as tecnologias e bibliotecas utilizadas no projeto estão:

- **Node.js**, utilizado como ambiente de execução;
- **Express**, utilizado na construção da aplicação web;
- **EJS**, utilizado para renderização das páginas;
- **Sequelize**, utilizado para comunicação com o banco de dados;
- **MySQL/MariaDB**, utilizado para armazenamento dos dados;
- **dotenv**, utilizado para gerenciamento de variáveis de ambiente;
- ferramentas utilizadas para **testes automatizados**.

A inclusão ou atualização de uma dependência deverá ser registrada no Git e passar pelo processo de revisão definido para o Shortz-App.

13.2 Atualização de Dependências

Antes de atualizar uma dependência importante, deverá ser verificado se a nova versão é compatível com o Shortz-App.

Após a atualização, os testes automatizados deverão ser executados para verificar se as funcionalidades continuam funcionando corretamente.

Por exemplo, uma atualização do **Sequelize** pode afetar a comunicação entre os Models do Shortz-App e o banco de dados. Por isso, a alteração não deverá ser realizada diretamente na versão principal sem passar pelas verificações definidas no projeto.

O fluxo recomendado será:

Atualização da dependência → Testes → Pull Request → CI → Revisão → Merge

13.3 Segurança das Dependências

As dependências utilizadas pelo Shortz-App também deverão ser verificadas quanto à existência de vulnerabilidades conhecidas.

Como prática DevOps, poderá ser incorporada ao pipeline de CI uma verificação automática das dependências.

Caso seja identificada uma vulnerabilidade relevante, a dependência deverá ser analisada e, quando necessário, atualizada para uma versão segura.

Isso permite que a segurança das bibliotecas utilizadas seja considerada durante o próprio processo de desenvolvimento, em vez de ser verificada somente depois que uma versão do Shortz-App estiver pronta.

14. Quadro-Resumo do Pipeline CI/CD

Resumo do fluxo padrão, do commit à produção:

Aspecto	**Aplicação no Shortz-App**
Proteção de informações sensíveis	Utilização de variáveis de ambiente para armazenar credenciais e configurações sensíveis, evitando seu versionamento no GitHub.
Segurança das dependências	Verificação das dependências utilizadas pelo projeto e atualização quando forem identificadas vulnerabilidades ou problemas de compatibilidade.
Autenticação	Verificação da sessão do usuário para permitir acesso às funcionalidades que exigem autenticação.
Controle de acesso	Garantia de que cada usuário possa acessar ou alterar somente os recursos aos quais possui permissão.
Testes de segurança	Testes para verificar acessos não autorizados, dados inválidos e tentativa de acesso a recursos de outros usuários.
Controle de mudanças	Alterações relacionadas à segurança devem passar por branch, Pull Request, revisão e testes antes do merge.
Automação	Inclusão de verificações de segurança no processo de CI, conforme a evolução do projeto.

Aspecto	**Aplicação no Shortz-App**
DevSecOps	Integração das práticas de segurança ao processo de desenvolvimento e entrega do Shortz-App.